

SECURITY REVIEW

FOR HYPERLEND



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	08
07	FINDINGS SUMMARY	09
08	DETAILED FINDINGS	10
L-01	Swap Adapters Wrap Entire Contract Native Balance Which Inflates Swap Output	10
I-01	Redundant <code>block.number</code> check in transient storage	12
I-02	Closing a Full Position Fails if the User Has Unrelated Debt in the Same Asset	13
I-03	StrategyManager Cannot Receive Native Token Refunds Despite Native Cleanup Branch	15
I-04	StrategyManagerFactory Registry Becomes Stale After Ownership Transfer	16
09	DISCLAIMER	17
10	ABOUT KANN AUDITS	18

AUDIT SUMMARY

HyperLend engaged Kann Audits to perform a security review of the HyperLend smart contracts. From 4 – 10 April, a team of 6 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

5
ISSUES

CRITICAL	-
HIGH	-
MEDIUM	-
LOW	01
INFORMATIONAL	04

ENGAGEMENT DETAILS

PREPARED FOR	HyperLend
PREPARED BY	Kann Audits
DATE REPORTED	26 May 2026
REVIEW PERIOD	4 – 10 April

PROTOCOL OVERVIEW

HyperLend is a high-performance lending protocol on Hyperliquid, built for capital efficiency. It offers real-time leverage, dynamic rates, and deep liquidity access. Built for traders, quants, and market makers in need of efficient lending.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	https://github.com/hyperlendx/looping-contracts/
AUDITED COMMIT	4f5deaf
FINAL COMMIT	4f5deaf

#	FILE
01	src/StrategyManagerFactory.sol
02	src/StrategyManager.sol
03	src/Looping.sol
04	src/periphery/GlueAdapter.sol
05	src/periphery/LiquidSwapAdapter.sol

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

SECURITY RESEARCHER

SA

sa1nt

SECURITY RESEARCHER

DE

dev_razkky

SECURITY RESEARCHER

DE

derastephh

SECURITY RESEARCHER

YU

yusufthebdev

SECURITY RESEARCHER

HE

heeze

SECURITY RESEARCHER

05 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01**Kickoff & Context**

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02**Scope & Preparation**

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03**AI First Scan**

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04**Expert Security Review**

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05**Findings & Proofs**

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06**Mitigation Review**

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL —	● HIGH —	● MEDIUM —	● LOW 01	● INFO 04
TOTAL 05 FIXED 01 ACKNOWLEDGED 04				

ID	TITLE	SEVERITY	STATUS
L-01	Swap Adapters Wrap Entire Contract Native Balance Which Inflates Swap Output	● LOW	● ACKNOWLEDGED
I-01	Redundant <code>block.number</code> check in transient storage	● INFORMATIONAL	● FIXED
I-02	Closing a Full Position Fails if the User Has Unrelated Debt in the Same Asset	● INFORMATIONAL	● ACKNOWLEDGED
I-03	StrategyManager Cannot Receive Native Token Refunds Despite Native Cleanup Branch	● INFORMATIONAL	● ACKNOWLEDGED
I-04	StrategyManagerFactory Registry Becomes Stale After Ownership Transfer	● INFORMATIONAL	● ACKNOWLEDGED

DETAILED FINDINGS

L-01

Swap Adapters Wrap Entire Contract Native Balance Which Inflates Swap Output

SEVERITY

● LOW

STATUS

● ACKNOWLEDGED

DESCRIPTION

In `swapExactTokensForTokensSupportingFeeOnTransferTokens`, when `tokenOut` is `WHYPE`, the adapter wraps the contract's entire native token balance into `WHYPE`:

```
if (address(this).balance > 0) {
    WHYPE.deposit{value: address(this).balance}();
}
```

As a result, any native tokens already present in the contract before the swap execution are also wrapped and included in the final `balanceOut` calculation together with the actual swap output.

This causes the output amount to be inflated since unrelated contract funds are accounted for as part of the current swap result.

```
function swapExactTokensForTokensSupportingFeeOnTransferTokens(
    ...
) external nonReentrant {
    ...

    (bool success, ) = gluex.call(gluexCallData);
    require(success, "GluexAdapter: gluex swap failed");

    if (address(this).balance > 0) {
        WHYPE.deposit{value: address(this).balance}();
    }

    uint256 balanceOut = IERC20(tokenOut).balanceOf(address(this));

    require(
        balanceOut ≥ amountOutMin,
        "GluexAdapter: minAmountOut > balanceOut"
    );

    IERC20(tokenOut).safeTransfer(to, balanceOut);
}
```

■ **IMPACT**

User receive more `baLance0ut` than intended since unrelated native funds already present in the contract are included in the swap result and transferred as part of the output amount.

■ **REMEDIATION** ● **ACKNOWLEDGED**

I-01

Redundant `block.number` check in transient storage

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

DESCRIPTION

Both `LiquidSwapAdapter` and `GlueXAdapter` store the current `block.number` in transient storage during `setSwapPath` and later verify that it matches during `swapExactTokens...`. The apparent intention is to ensure that the swap path is only valid if it was set within the same transaction as the swap execution.

```
function setSwapPath(
    address tokenIn,
    address tokenOut,
    bytes calldata gluexData
) external {
    // Generate unique slots for this token pair in transient storage
    bytes32 baseSlot = keccak256 (abi.encodePacked(tokenIn, tokenOut));
    bytes32 blockSlot = keccak256 (abi.encodePacked(baseSlot, "block"));

    assembly {
        tstore(blockSlot, number())
        tstore(baseSlot, gluexData.length)
    }

    // Store data in chunks of 32 bytes
    uint256 length = gluexData .length;
    for (uint256 i = 0; i < length; i += 32) {
        bytes32 chunk;
        assembly {
            chunk := calldataload (add(gluexData.offset, i))
            tstore(add(baseSlot , add(1, div(i, 32))), chunk)
        }
    }
}
```

However, this additional `block.number` validation is redundant in the context of EIP-1153 transient storage. Under EIP-1153, transient storage is automatically cleared at the end of every transaction, not at the end of a block. As a result, any value written via `tstore` in one transaction cannot persist into a subsequent transaction, even if it occurs within the same block.

Therefore, the safety property the check attempts to enforce ensuring same transaction usage is already guaranteed by the EVM's transaction-scoped lifetime of transient storage. The comparison against `block.number` does not strengthen this guarantee and does not provide additional security beyond what EIP-1153 inherently enforces.

REMEDIATION

● FIXED

I-02

Closing a Full Position Fails if the User Has Unrelated Debt in the Same Asset

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

DESCRIPTION

In `Looping.sol`, users can close a leveraged position by providing the flashloan parameters and setting `withdrawAmount` to `type(uint256).max`. This signals the contract to fully unwind the position by reading the user's current yield token and debt token balances.

```
function _executeClosePosition(bytes memory params, address debtAsset) internal {
    (
        ...
    ) = abi.decode(...);

    IERC20 hYieldToken = IERC20(
        IPool(msg.sender).getReserveData( yieldAsset).aTokenAddress
    );

    // close full position if withdrawAmount == maxUint256
    if (withdrawAmount == type(uint256).max) {
        IERC20 debtDebtToken = IERC20(
            IPool(msg.sender)
                .getReserveData(debtAsset)
                .variableDebtTokenAddress
        );

        repaymentAmount = debtDebtToken.balanceOf(user);
        withdrawAmount = hYieldToken.balanceOf(user);
    }

    ...
}
```

However, `debtDebtToken.balanceOf(user)` returns the user's global debt balance for that specific asset across the entire lending pool. If the user has other active borrows using the same debt token (backed by different collateral assets outside of this specific looping strategy), `repaymentAmount` will be set to a value higher than the `flashloanAmount`. Because the `Looping` contract only holds the exact amount of `debtAsset` it flashloaned, the subsequent call to `IPool.repay` will revert due to insufficient balance.

If user opens first USDC/WETH position by supplying WETH then USDC/WBTC. Debt token is the same. If user wants to exit with the full WETH position, we wouldn't be able to since `repaymentAmount = debtDebtToken.balanceOf(user)`; will take the debt balance from both positions.

To mitigate this we could specify the exact `repaymentAmount` amount instead of putting `uint256 max`. Or we could flashloan more amount so it repays all the debt but its not more than the collateral that will be swapped to the `debtAsset`. In this situation the flashloan amount < WETH. Then the second position is now without any debt so user can just call withdraw

■ **REMEDIATION**

● **ACKNOWLEDGED**

I-03

StrategyManager Cannot Receive Native Token Refunds Despite Native Cleanup Branch

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ RELEVANT CODE

```
if (tokens[i] == address (0)){
    (bool sent,) = owner().call{value: address(this).balance}("");
    require(sent, " cleanOutTokens: failed to send native");
}
```

■ DESCRIPTION

StrategyManager contains logic for cleaning out native balances when `address(0)` is passed, which implies native-asset handling was considered. However, the contract does not implement `receive()` or a payable `fallback()`, so standard native refunds or direct native transfers to the contract revert.

■ REMEDIATION

● ACKNOWLEDGED

I-04

StrategyManagerFactory Registry Becomes Stale After Ownership Transfer

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

StrategyManagerFactory indexes strategy managers by the address that created them, but StrategyManager ownership is transferable after deployment.

Once a manager is transferred to a new owner, the factory registry is not updated. The original creator remains recorded as the owner of that tuple while the real new owner is invisible to the registry.

■ REMEDIATION

● ACKNOWLEDGED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits